Kontextsensitive, lokale KI-Assistenz zur automatisierten Barrierefreiheitsanalyse von Rich-Client-Webanwendungen mittels Tool-Reports, Playwright-Interaktionen und Codeanalyse

Bachelorarbeit

vorgelegt am 11.05.2026

Fakultät Wirtschaft und Gesundheit

Studiengang Wirtschaftsinformatik

Kurs WWI2023

von

Joel Yerai Martinez Campo

Betreuung in der Ausbildungsstätte:

BITBW
Ilko Hoffman
Betreuer der Bachelorarbeit

Unterschrift

DHBW Stuttgart:

Sascha Alexander Ströbel

LaTeX-Vorlage für Projekt-, Seminar- und Bachelorarbeiten

Dies wird meine Bachelorarbeit

Inhaltsverzeichnis

Abkürzungsverzeichnis **IV**

Abbildungsverzeichnis **V**

Tabellenverzeichnis **VI**

1 Einleitung **1**
1.1 Motivation und Kontext 1
1.2 Problemstellung 3
1.3 Zielsetzung und Forschungsfragen 4
1.4 Aufbau der Arbeit 5

2 Theoretische Grundlagen **6**
2.1 Barrierefreiheit von Webanwendungen 6
2.2 Automatisierte Barrierefreiheitsanalyse 6
2.3 Rich-Client-Webanwendungen und Testautomatisierung 6
2.4 KI-Assistenzsysteme für Softwareanalyse und Handlungsempfehlungen 6

3 Methodik **7**
3.1 Wissenschaftliche Methode Fallstudienanalyse 7
3.2 Untersuchungsgegenstand und Scope-Abgrenzung 7
3.3 Evaluationsdesign und Kriterien 7

4 Analyse und Anforderungen **8**
4.1 Analyse der Tool-Outputs und typischer Findings 8
4.2 Analyse von Interaktionsdaten aus Playwright 8
4.3 Analyse der Codebasis als Kontextquelle 8
4.4 Datenschutz & Sicherheitsanforderungen 8

5 Konzeption des Assistenzsystems **9**
5.1 Zielarchitektur und Datenpipeline 9
5.2 Modell der Kontextualisierung & Priorisierung 9
5.3 Ausgabeformat entwicklernahe To-Do-Liste 9

6 Implementierung des Proof of Concept **10**
6.1 Technische Umsetzung 10
6.2 Beispielabläufe und Ergebnisse am Untersuchungsobjekt 10

7 Evaluation & Diskussion **11**
7.1 Ergebnisse entlang der Kriterien und Limitationen 11
7.2 Handlungsempfehlungen und Ausblick 11

8 Fazit **12**

Anhang **13**

Literaturverzeichnis **16**

Abkürzungsverzeichnis

BFSG Barrierefreiheitsstärkungsgesetz

BITBW IT Baden-Württemberg

BITV Barrierefreie-Informationstechnik-Verordnung

DOM Document Object Model

EAA European Accessibility Act

LLM Large Language Model

TF Teilfragen

WCAG Web Content Accessibility Guidelines

Abbildungsverzeichnis

Tabellenverzeichnis

1 Konzeptmatrix (Hauptmatrix, hoch priorisierte Studien) 7
2 Konzeptmatrix (vollständige Übersicht) 15

1 Einleitung

Digitale Anwendungen sind für Verwaltung, Wirtschaft und Gesellschaft zu einer zentralen Infrastruktur geworden. Damit alle Menschen, unabhängig von individuellen Einschränkungen, gleichberechtigten Zugang zu Informationen und Dienstleistungen erhalten, gewinnt Web-Barrierefreiheit zunehmend an Bedeutung. In der Praxis zeigt sich jedoch eine Diskrepanz zwischen normativen Anforderungen (z. B. Web Content Accessibility Guidelines (WCAG)-basierte Kriterien)[1] und der tatsächlichen Umsetzung in Softwareprojekten: Barrierefreiheit wird häufig spät berücksichtigt, Prüfungen erfolgen punktuell und Korrekturen sind zeitaufwendig. Besonders in gewachsenen Systemlandschaften mit Legacy-Anteilen und knappen Entwicklungsressourcen entstehen so wiederkehrende Qualitätsdefizite.

Parallel dazu haben sich Large Language Model (LLM) als leistungsfähige Werkzeuge zur Unterstützung von Softwareentwicklung etabliert. Sie helfen, Aufgaben wie Code-Analyse, Refactoring oder die Erstellung technischer Empfehlungen zu beschleunigen. Für die Barrierefreiheit ist dies besonders relevant: Hier müssen nicht nur Fehler erkannt, sondern auch korrekt, standardkonform und kontextsensitiv behoben werden. Gleichzeitig bestehen in vielen Organisationen, insbesondere in sensiblen Umgebungen, strikte Vorgaben an Datenschutz und Informationssicherheit, wodurch Cloud-basierte Assistenzsysteme häufig nicht eingesetzt werden dürfen. Daraus ergibt sich die zentrale Herausforderung, LLM-Unterstützung so zu gestalten, dass sie lokal, nachvollziehbar und qualitätsgesichert in bestehende Entwicklungsprozesse integriert werden kann.

Diese Arbeit untersucht daher, wie ein LLM- bzw. agentenbasierter Ansatz zur automatisierten Erkennung und Behebung von Barrierefreiheitsproblemen in Webanwendungen konzipiert und prototypisch umgesetzt werden kann.[2] Im Fokus steht ein Workflow, der automatisierte Tests (z. B. via Browser-Automation und Accessibility-Linter) mit einer LLM-gestützten Analyse und Änderungsvorschlägen verbindet und die Änderungen anschließend erneut testet. Ziel ist es, den manuellen Aufwand zu reduzieren, die Konsistenz der Ergebnisse zu erhöhen und Barrierefreiheit als kontinuierlichen Bestandteil der Softwarequalität zu verankern.

1.1 Motivation und Kontext

Rund 1,3 Milliarden Menschen (16% der Weltbevölkerung) leben mit einer Form von Behinderung.[3] Für diese Personengruppe ist der Zugang zu digitalen Angeboten keine bloße Komfortfrage, sondern eine Voraussetzung für gesellschaftliche Teilhabe in nahezu allen Lebensbereichen: von der Kommunikation mit Behörden über den Zugang zu Bildung und Gesundheitsinformationen bis hin zur Teilnahme am wirtschaftlichen Leben. Das Recht auf barrierefreien Zugang zu Informations- und Kommunikationstechnologien ist völkerrechtlich in der UN-

[1] Vgl. World Wide Web Consortium (W3C) 2026
[2] Vgl. Bassi et al. 2025, S.297 ff.
[3] Vgl. World Health Organization 2026

Behindertenrechtskonvention verankert und hat sich in den vergangenen Jahren zunehmend auch in verbindlichem nationalen und europäischen Recht niedergeschlagen.[4]

Auf europäischer Ebene verpflichtet der European Accessibility Act (EAA) die Mitgliedstaaten, Barrierefreiheitsanforderungen für digitale Produkte und Dienstleistungen durchzusetzen.[5] In Deutschland wurde diese Richtlinie durch das Barrierefreiheitsstärkungsgesetz (BFSG) in nationales Recht überführt, das seit dem 28. Juni 2025 auch für weite Teile der Privatwirtschaft gilt.[6] Ergänzt wird dieser Rahmen durch die Barrierefreie-Informationstechnik-Verordnung (BITV), die für Behörden und öffentliche Stellen bereits seit Jahren verbindliche Vorgaben formuliert.[7][8]

Trotz dieser eindeutigen Rechtslage ist die praktische Umsetzung barrierefreier Webanwendungen nach wie vor unzureichend. Die jährlich durchgeführte WebAIM-Million-Studie, die die Startseiten der meistbesuchten Webseiten weltweit analysiert, stellte 2025 fest, dass 94,8% der untersuchten Seiten nachweisbare Verstöße gegen die WCAG aufweisen.[9] Damit bleibt Barrierefreiheit in der Entwicklungspraxis trotz gesetzlicher Verpflichtung systematisch unterrepräsentiert. Im Vergleich mit den letzten Jahren haben wir seit 2019 (97,8%) lediglich eine Verbesserung um 3,1 Prozentpunkte.[10]

Die IT Baden-Württemberg (BITBW) nimmt als zentraler IT-Dienstleister der Landesverwaltung Baden-Württemberg in diesem Kontext eine besondere Stellung ein. Am 1. Juli 2015 auf Grundlage des gleichnamigen BITBW-Gesetzes gegründet, löste sie das ehemalige Informatikzentrum Landesverwaltung ab und fungiert seither als dem Innenministerium untergeordnete Landesoberbehörde.[11] Zu ihren Kunden zählen Ministerien, Polizeibehörden, Kommunen sowie Stiftungen des Öffentlichen Rechts; ihre Aufgaben umfassen die „Bereitstellung, Betrieb und Ausbau der zentralen informationstechnischen Infrastruktur für die Landesverwaltung [...]", die „Sicherstellung der Informationssicherheit [...]“ sowie die Beschaffung von IT-Produkten und Lizenzen.[12]

Als Entwicklerin und Betreiberin öffentlicher Webanwendungen ist sie dabei gesetzlich zur Einhaltung der BITV und dem BFSG verpflichtet und trägt unmittelbare Verantwortung dafür, dass ihre Systeme für alle Bürgerinnen und Bürger zugänglich sind, unabhängig den jeweiligen Einschränkungen die diese Bürger haben könnten. Sie bildet damit den praktischen Rahmen der vorliegenden Arbeit.

[4] Vgl. United Nations Human Rights 2026
[5] Vgl. European Comission 2026
[6] Vgl. Barrierefreiheitsstärkungsgesetz 2026
[7] Vgl. Heinemann 2024, S. 281
[8] Vgl. Bundesministerium für Digitales und Staatsmodernisierung 2025
[9] Vgl. WebAIM 2025a
[10] Vgl. WebAIM 2019
[11] Vgl. BITBW 2021
[12] Landesrecht BW 2015

1.2 Problemstellung

Die mangelhafte Umsetzung von Barrierefreiheit in Softwareprojekten lässt sich nicht allein auf fehlende Motivation oder Ressourcen zurückführen. Ein wesentlicher Treiber sind die Schwächen der verfügbaren Werkzeuge und Tools: Automatisierte Barrierefreiheitsanalyse-Tools wie axe[13], WAVE[14] oder Lighthouse[15] erkennen zwar syntaktische Verstöße, stoßen jedoch bei semantischen Problemen an ihre Grenzen. Ob ein vorhandener Alternativtext den Bildinhalt tatsächlich angemessen beschreibt, oder ob eine Schaltflächenbeschriftung für Screenreader-Nutzerinnen und -Nutzer verständlich ist, entzieht sich der regelbasierten Prüflogik dieser Werkzeuge.

Empirisch belegt wird diese Schwäche durch das Mutation-Testing-Framework **Ma11y**: In einer systematischen Evaluation mit 25 gezielt eingebrachten Barrierefreiheits-Defekten erkannte kein einzelnes der sechs untersuchten Tools mehr als 52% der injizierten Fehler. Somit blieben im Durchschnitt nahezu die Hälfte aller Defekte unentdeckt.[16] Besonders aufschlussreich ist dabei die Differenzierung nach Verstoßtypen: Während syntaktische Defekte zu 54 Prozent erkannt wurden, lag die Erkennungsrate bei semantischen Verstößen lediglich bei 26 Prozent.[17] Sechs der 25 Defekttypen - darunter die Manipulation von Seitentiteln und Alternativtexten - wurden von keinem der untersuchten Tools erkannt.

Eine zweite strukturelle Schwäche betrifft die Charakteristik moderner Webanwendungen. Rich-Client-Anwendungen auf Basis von Frameworks wie React erzeugen ihren Inhalt größtenteils clientseitig und verändern den Document Object Model (DOM) dynamisch in Abhängigkeit von Nutzerinteraktionen. Barrieren entstehen häufig erst in bestimmten Zuständen: wenn ein Modalfenster geöffnet wird, eine Fehlermeldung erscheint oder ein Dropdown-Menü aktiviert ist. Rein statische Analysen - oder Analysen, die ausschließlich den initialen Seitenladezustand prüfen - erfassen diese zustandsabhängigen Barrieren nicht.[18]

Jüngere Forschungsarbeiten zeigen, dass LLMs geeignet sind, die Grenzen regelbasierter Tools zu überwinden. So erreicht das System **AccessGuru**[19] durch den Einsatz von GPT-4o eine Reduktion des Violation Scores um bis zu 84% , und **GenA11y**[20] erzielt bei der Erkennung von Barrierefreiheitsproblemen eine Precision von 94,5%. Diese Ansätze beschränken sich jedoch auf statisches HTML. Keiner der bekannten Ansätze verknüpft Tool-Reports, Playwright-Interaktionsdaten und den Quellcode der geprüften Komponenten zu einem integrierten Analysesystem.[21] Der Schritt von der Fehlererkennung zu einer entwicklernahen, direkt umsetzbaren Handlungsempfehlung - idealerweise mit Bezug zur konkreten Quellcodestelle - wird in der Literatur zwar als offenes Problem benannt,[22] bleibt aber bislang ungelöst.

[13] Vgl. axe 2026
[14] Vgl. WebAIM 2025b
[15] Vgl. Google 2025
[16] Vgl. Tafreshipour et al. 2024, S. 1-12
[17] Vgl. Tafreshipour et al. 2024, S. 9
[18] Vgl. Bassi et al. 2025, S. 1-10
[19] Vgl. Fathallah, Hernández, Staab 2025
[20] Vgl. He, Huq, Malek 2025
[21] Vgl. Fathallah, Hernández, Staab 2025, S. 18
[22] Vgl. He, Huq, Malek 2025, S. 7 f.

Daraus ergibt sich eine klar umrissene Lücke: Es fehlt ein Assistenzsystem, das (a) die dynamischen Zustände einer React-Anwendung durch automatisierte Browser-Interaktionen erfasst, (b) die Ergebnisse automatisierter Tool-Reports mit dem relevanten Quellcode verknüpft und (c) auf dieser Grundlage priorisierte, entwicklernahe Handlungsempfehlungen erzeugt - lokal ausführbar und ohne den Transfer sensibler Anwendungsdaten an externe Dienste.

1.3 Zielsetzung und Forschungsfragen

Ziel dieser Bachelorarbeit ist die Konzeption und prototypische Umsetzung einer lokal ausführbaren, kontextsensitiven Assistenzlösung zur Barrierefreiheitsanalyse von Rich-Client-Webanwendungen. Als Untersuchungsobjekt dient eine React-basierte Webanwendung aus dem Umfeld der BITBW. Der Proof of Concept soll automatisierte Barrierefreiheits-Reports, Interaktionsdaten aus Playwright-Tests sowie ausgewählte Informationen aus der Codebasis in einer Datenpipeline zusammenführen. Auf dieser Grundlage werden priorisierte, entwicklernahe Handlungsempfehlungen abgeleitet und strukturiert aufbereitet. Die Arbeit liefert drei konkrete Beiträge: Erstens einen strukturierten Überblick darüber, wie unterschiedliche Datenquellen der Barrierefreiheitsanalyse systematisch kombiniert und interpretiert werden können. Zweitens einen Proof of Concept, der das Vorgehen exemplarisch demonstriert. Drittens eine qualitative Evaluation anhand definierter Kriterien, die aus den Anforderungen und der einschlägigen Fachliteratur abgeleitet werden.

Die übergeordnete Forschungsfrage dieser Arbeit lautet:

Inwieweit kann ein kontextsensitives KI-Assistenzsystem, das Tool-Reports, Playwright-Interaktionsdaten und Quellcode kombiniert, Barrierefreiheitsverstöße in React-basierten Webanwendungen automatisiert erkennen und entwicklernahe Handlungsempfehlungen generieren?

Diese übergeordnete Frage wird durch zwei konkretisierende Teilfragen (TF) operationalisiert:

- TF1: Welche Typen von Barrierefreiheitsverstößen - syntaktisch, semantisch und layoutbezogen - lassen sich durch den gewählten Ansatz zuverlässig erkennen, und wo liegen die methodischen Grenzen?
- TF2: Inwiefern verbessert die Einbeziehung von Quellcode und Playwright-Interaktionsdaten die Qualität der generierten Handlungsempfehlungen gegenüber rein DOM-basierten Ansätzen?

TF1 adressiert dabei das Erkennungsproblem und erlaubt eine differenzierte Aussage über die Leistungsfähigkeit und Grenzen des Systems entlang der Verstoß-Taxonomie von Fathallah et al.[23][24] TF2 greift den in der Literatur identifizierten Mehrwert der Quellcode-Kontextualisierung auf und überprüft ihn im Rahmen einer qualitativen Evaluation am gewählten Untersuchungsobjekt.

[23] Vgl. Fathallah, Hernández, Staab 2025
[24] Vgl. Šmite et al. 2012, S. 105-153

1.4 Aufbau der Arbeit

Die vorliegende Arbeit gliedert sich in neun Kapitel. Im Anschluss an die Einleitung werden in Kapitel 2 die theoretischen Grundlagen erarbeitet: Barrierefreiheit im Web, die WCAG-Standards und der rechtliche Rahmen, die Charakteristika von Rich-Client-Webanwendungen sowie Grundlagen zu Large Language Models und Prompt-Engineering-Strategien.

Kapitel 3 gibt einen systematischen Überblick über den Stand der Forschung. Die relevanten Arbeiten werden thematisch gegliedert - nach LLM-basierter Erkennung, Code-Korrektur, hybriden Pipelines und Evaluation - und in einer Konzeptmatrix nach Webster und Watson zusammengefasst.[25] Das Kapitel schließt mit einer expliziten Einordnung des Beitrags der vorliegenden Arbeit.

Kapitel 4 legt das methodische Vorgehen dar. Es beschreibt die Wahl der Fallstudienanalyse als wissenschaftliche Methode, definiert den Untersuchungsgegenstand und entwirft das Evaluationsdesign. Kapitel 5 analysiert die verfügbaren Datenquellen - Tool-Reports, Playwright-Interaktionsdaten und Codebasis - und leitet daraus funktionale und nicht-funktionale Anforderungen ab.

Auf Basis dieser Anforderungen wird in Kapitel 6 das Assistenzsystem konzipiert: Systemarchitektur, Datenpipeline, Prompt-Strategie und Ausgabeformat werden begründet entworfen. Kapitel 7 beschreibt die prototypische Implementierung des Proof of Concept und illustriert die Systemfunktionalität anhand von Beispielabläufen für syntaktische, semantische und dynamisch erzeugte Barrierefreiheitsprobleme.

Kapitel 8 präsentiert und diskutiert die Evaluationsergebnisse, beantwortet die Forschungsfragen und reflektiert die Limitationen des Ansatzes. Das abschließende Kapitel 9 fasst die Kernaussagen zusammen, ordnet den Beitrag der Arbeit in die Forschungslandschaft ein und formuliert Empfehlungen für weiterführende Untersuchungen.

[25] Vgl. Webster, Watson 2002

2 Theoretische Grundlagen

2.1 Barrierefreiheit von Webanwendungen

2.2 Automatisierte Barrierefreiheitsanalyse

2.3 Rich-Client-Webanwendungen und Testautomatisierung

2.4 KI-Assistenzsysteme für Softwareanalyse und Handlungsempfehlungen

3 Methodik

3.1 Wissenschaftliche Methode Fallstudienanalyse

Studie		**Konzepte**												
		Ziel			**Datenquellen**			**KI-/LLM-Ansatz**			**Evaluation**			**Ko**
Autor(en) & Jahr	**System**	WCAG-Konf. prüfen	Barrieren korrigieren	Entwickler unterstützen	Tool-Reports (axe etc.)	Interaktionsdaten (Playwright)	Quellcode-Analyse	LLM-basiert	Kontextsensitiv / RAG	Lokal ausführbar	Automatisierte Eval.	Qualitative Eval.	Nutzerstudie	Rich-Client / SPA
Hoch priorisierte Studien														
He et al. (2025)	GenA11y	x			x			x			x			
Fathallah et al. (2025)	AccessGuru	x	x	x	x	x		x			x			
Bassi et al. (2025)	LLM Auditing	x	x	x				x				x		
Paternò et al. (2025)	TeVal	x		x	x			x	x			x		
Tafreshipour et al. (2024)	Ma11y	x			x	x					x			
Martinez Campo (2026)	**ACE**	x	x	x	x	x	x	x	x	x		x		x

Tab. 1: Konzeptmatrix nach Webster Watson[26]: Hoch priorisierte Studien im Vergleich zum eigenen Proof of Concept. **x** = Konzept vorhanden/adressiert; leer = nicht adressiert. Die vollständige Matrix aller analysierten Studien findet sich in Anhang 1.

3.2 Untersuchungsgegenstand und Scope-Abgrenzung

3.3 Evaluationsdesign und Kriterien

4 Analyse und Anforderungen

4.1 Analyse der Tool-Outputs und typischer Findings

4.2 Analyse von Interaktionsdaten aus Playwright

4.3 Analyse der Codebasis als Kontextquelle

4.4 Datenschutz & Sicherheitsanforderungen

5 Konzeption des Assistenzsystems

5.1 Zielarchitektur und Datenpipeline

5.2 Modell der Kontextualisierung & Priorisierung

5.3 Ausgabeformat entwicklernahe To-Do-Liste

6 Implementierung des Proof of Concept

6.1 Technische Umsetzung

6.2 Beispielabläufe und Ergebnisse am Untersuchungsobjekt

7 Evaluation & Diskussion

7.1 Ergebnisse entlang der Kriterien und Limitationen

7.2 Handlungsempfehlungen und Ausblick

8 Fazit

Anhang

Anhangverzeichnis

Anhang 1 Erweiterte Konzeptmatrix 14

Anhang 1: Erweiterte Konzeptmatrix

Studie		**Konzepte**												
		Ziel			**Datenquellen**			**KI-/LLM-Ansatz**			**Evaluation**			**Ko**
Autor(en) & Jahr	**System**	WCAG-Konf. prüfen	Barrieren korrigieren	Entwickler unterstützen	Tool-Reports (axe etc.)	Interaktionsdaten (Playwright)	Quellcode-Analyse	LLM-basiert	Kontextsensitiv / RAG	Lokal ausführbar	Automatisierte Eval.	Qualitative Eval.	Nutzerstudie	Rich-Client / SPA
Hoch priorisierte Studien														
He et al. (2025)	GenA11y	x			x			x			x			
Fathallah et al. (2025)	AccessGuru	x	x	x	x	x		x			x			
Bassi et al. (2025)	LLM Auditing	x	x	x				x				x		
Paternò et al. (2025)	TeVal	x		x	x			x	x			x		
Tafreshipour et al. (2024)	Ma11y	x			x	x					x			
Fernández-Navarro & Chicano (2025)	LLM Remediation	x	x	x	x		x	x			x			x
Mittel priorisierte Studien														
Yu et al. (2025)	GenAI Screen Reader	x	x		x		x	x	x		x	x	x	
Huang et al. (2024)	ACCESS	x	x	x	x	x		x			x			
Pool (2023)	Testaro	x			x	x					x			
Mowar et al. (2024)	Copilot Study	x		x				x					x	
Abu Doush & Kassem (2024)	AI Code Study	x	x	x				x			x			
Guriță & Vatavu (2025)	AI UI Study	x						x			x			
Kodithuwakku & Wick. (2025)	Tool Eval.	x			x						x			
Manca et al. (2023)	Transparency	x		x	x							x	x	
Jiang & Nam (2025)	Cursor Rules			x			x	x				x		
Gubbi Mohanbabu & Pavel (2024)	Context-Aware Img.	x	x					x	x		x	x	x	
Campoverde-Molina & Luján-Mora (2025)	AI A11y SMS	x	x	x	x			x				x		
Leedy et al. (2025)	GenAI Builder Eval.	x			x			x			x	x		
Patel & Melcer (2025)	AIDA Study	x		x				x				x	x	
Aljedaani et al. (2024)	ChatGPT A11y Code	x	x	x			x	x			x			
Niedrig priorisierte Studien														
Abou-Zahra et al. (2018)	AI for A11y	x						x				x		
Delnevo et al. (2024)	LLM Interaction	x	x					x				x		x
Draffan et al. (2020)	Web2Access+AI	x			x			x				x		
Eigener Proof of Concept														
Martinez Campo (2025)	**ACE**	x	x	x	x	x	x	x	x	x		x		x

Tab. 2: Konzeptmatrix nach Webster Watson[27]: Vollständige Übersicht aller analysierten Studien im Vergleich zum eigenen Proof of Concept. **x** = Konzept vorhanden/adressiert; leer = nicht adressiert.

Literaturverzeichnis

axe (2026): *dequelabs/axe-core*. original-date: 2015-06-10T15:26:45Z. URL: https://github.com/dequelabs/axe-core (Abruf: 24. 02. 2026).

Barrierefreiheitsstärkungsgesetz (2026): BFSG (Barrierefreiheitsstärkungsgesetz). URL: https://bfsg-gesetz.de/ (Abruf: 24. 02. 2026).

Bassi, Barry; Delnevo, Giovanni; Franco, Mirko; Gaggi, Ombretta; Gatto, Salvatore; Mirri, Silvia; Olaiya, Kelvin (2025): An Assessment of LLM-Based Auditing and Validation for Web Accessibility. In: *Proceedings of the 2025 International Conference on Information Technology for Social Good*. GoodIT '25. New York, NY, USA: Association for Computing Machinery, S. 297–305. ISBN: 979-8-4007-2089-5. DOI: 10.1145/3748699.3749805. URL: https://dl.acm.org/doi/10.1145/3748699.3749805 (Abruf: 24. 02. 2026).

BITBW (2021): 6 Jahre BITBW. URL: https://bitbw.de/neuigkeiten/artikel/6-jahre-bitbw-1 (Abruf: 12. 07. 2025).

Bundesministerium für Digitales und Staatsmodernisierung (2025): Barrierefreie-Informationstec Verordnung (BITV 2.0). URL: https://www.barrierefreiheit-dienstekonsolidierung.bund.de/Webs/PB/DE/gesetze-und-richtlinien/bitv2-0/bitv2-0-node.html (Abruf: 24. 02. 2026).

European Comission (2026): European accessibility act (EAA). URL: https://commission.europa.eu/strategy-and-policy/policies/justice-and-fundamental-rights/disability/european-accessibility-act-eaa_en (Abruf: 2026).

Fathallah, Nadeen; Hernández, Daniel; Staab, Steffen (2025): AccessGuru: Leveraging LLMs to Detect and Correct Web Accessibility Violations in HTML Code. In: *Proceedings of the 27th International ACM SIGACCESS Conference on Computers and Accessibility*. ASSETS '25. New York, NY, USA: Association for Computing Machinery, S. 1–22. ISBN: 979-8-4007-0676-9. DOI: 10.1145/3663547.3746360. URL: https://dl.acm.org/doi/10.1145/3663547.3746360 (Abruf: 24. 02. 2026).

Google (2025): Lighthouse : Chrome for developers. URL: https://developer.chrome.com/docs/lighthouse/overview (Abruf: 06. 10. 2025).

He, Ziyao; Huq, Syed Fatiul; Malek, Sam (2025): Enhancing Web Accessibility: Automated Detection of Issues with Generative AI. In: *Proc. ACM Softw. Eng.* 2 (FSE), FSE101:2264–FSE101:2287. DOI: 10.1145/3729371. URL: https://dl.acm.org/doi/10.1145/3729371 (Abruf: 24. 02. 2026).

Heinemann, Daniela (2024): „Barrierefreiheit". In: *Digitalisierte Verwaltung - Vernetztes E-Government*. Hrsg. von Margrit Seckelmann. Berlin: Erich Schmidt Verlag GmbH & Co. KG, S. 469–488. ISBN: 978-3-503-23763-0. DOI: 10.37307/b.978-3-503-23763-0.15. URL: https://doi.org/10.37307/b.978-3-503-23763-0.15 (Abruf: 24. 02. 2026).

Landesrecht BW (2015): Gesetz zur Errichtung der Landesoberbehörde BITBW. URL: https://www.landesrecht-bw.de/bsbw/document/jlr-ITErGBWV3P2 (Abruf: 12. 07. 2025).

Šmite, Darja; Wohlin, Claes; Galviņa, Zane; Prikladnicki, Rafael (2012): An empirically based terminology and taxonomy for Global Software Engineering. In: *Empirical Software Engineering* 19.1, S. 105–153. DOI: 10.1007/s10664-012-9217-9.

Tafreshipour, Mahan; Deshpande, Anmol; Mehralian, Forough; Ahmed, Iftekhar; Malek, Sam (2024): Ma11y: A Mutation Framework for Web Accessibility Testing. In: *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. ISSTA 2024. New York, NY, USA: Association for Computing Machinery, S. 100–111. ISBN: 979-8-4007-0612-7. DOI: 10.1145/3650212.3652113. URL: https://dl.acm.org/doi/10.1145/3650212.3652113 (Abruf: 24. 02. 2026).

United Nations Human Rights (2026). URL: https://www.ohchr.org/en/instruments-mechanisms/instruments/convention-rights-persons-disabilities (Abruf: 28. 02. 2026).

WebAIM (2019): The WebAIM Million - 2019 report on the accessibility of the top 1,000,000 home pages. URL: https://webaim.org/projects/million/2019 (Abruf: 24. 02. 2026).

WebAIM (2025a): The WebAIM Million - 2025 report on the accessibility of the top 1,000,000 home pages. URL: https://webaim.org/projects/million/ (Abruf: 24. 02. 2026).

WebAIM (2025b): Wave Web Accessibility Evaluation Tools. URL: https://wave.webaim.org/ (Abruf: 06. 10. 2025).

Webster, Jane; Watson, Richard T. (2002): Analyzing the Past to Prepare for the Future: Writing a Literature Review. In: *MIS Quarterly* 26.2, S. 2–6.

World Health Organization (2026). URL: https://www.who.int/health-topics/disability#tab=tab_1 (Abruf: 28. 02. 2026).

World Wide Web Consortium (W3C) (2026): Web Content Accessibility Guidelines (WCAG) 2.2. URL: https://www.w3.org/TR/WCAG22/ (Abruf: 24. 02. 2026).

Erklärung zur Verwendung generativer KI-Systeme

Bei der Erstellung der eingereichten Arbeit habe ich auf künstlicher Intelligenz (KI) basierte Systeme benutzt:

☒ ja

☐ nein[28]

Falls ja: Die nachfolgend aufgeführten auf künstlicher Intelligenz (KI) basierten Systeme habe ich bei der Erstellung der eingereichten Arbeit benutzt:

1.

2.

3. ...

Ich erkläre, dass ich

- mich aktiv über die Leistungsfähigkeit und Beschränkungen der oben genannten KI-Systeme informiert habe,[29]
- die aus den oben angegebenen KI-Systemen direkt oder sinngemäß übernommenen Passagen gekennzeichnet habe,
- überprüft habe, dass die mithilfe der oben genannten KI-Systeme generierten und von mir übernommenen Inhalte faktisch richtig sind,
- mir bewusst bin, dass ich als Autorin bzw. Autor dieser Arbeit die Verantwortung für die in ihr gemachten Angaben und Aussagen trage.

Die oben genannten KI-Systeme habe ich wie im Folgenden dargestellt eingesetzt:

Arbeitsschritt in der wissenschaftlichen Arbeit	**Eingesetzte(s) KI-System(e)**	**Beschreibung der Verwendungsweise**

[28] Die Erklärung ist in jedem Fall zu unterzeichnen, auch wenn Sie keine KI-Systeme genutzt haben und Ihr Kreuz bei „nein“ gesetzt haben.

[29] U.a. gilt es hierbei zu beachten, dass an KI weitergegebene Inhalte ggf. als Trainingsdaten genutzt und wiederverwendet werden. Dies ist insb. für betriebliche Aspekte als kritisch einzustufen.

(Ort, Datum) (Unterschrift)

Erklärung

Ich versichere hiermit, dass ich die vorliegende Arbeit mit dem Thema: *Kontextsensitive, lokale KI-Assistenz zur automatisierten Barrierefreiheitsanalyse von Rich-Client-Webanwendungen mittels Tool-Reports, Playwright-Interaktionen und Codeanalyse* selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

(Ort, Datum) (Unterschrift)